



Security Audit

tBTC (Defi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	19
Conclusion	20
Our Methodology	21
Disclaimers	23
About Hashlock	24

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The tBTC team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

tBTC is a decentralized, permissionless protocol that enables Bitcoin (BTC) holders to utilize their assets within the Starknet ecosystem. By minting tBTC, an ERC-20 token backed 1:1 by BTC, users can engage with decentralized finance (DeFi) applications on Starknet without relying on centralized intermediaries. Developed as an open-source project and operated by the Threshold Network, tBTC leverages threshold cryptography and a decentralized network of nodes to securely bridge BTC to Starknet and other EVM-compatible networks. This integration allows BTC holders to access a broader range of financial services, including lending, borrowing, and trading, thereby enhancing the utility of their assets.

Project Name: tBTC

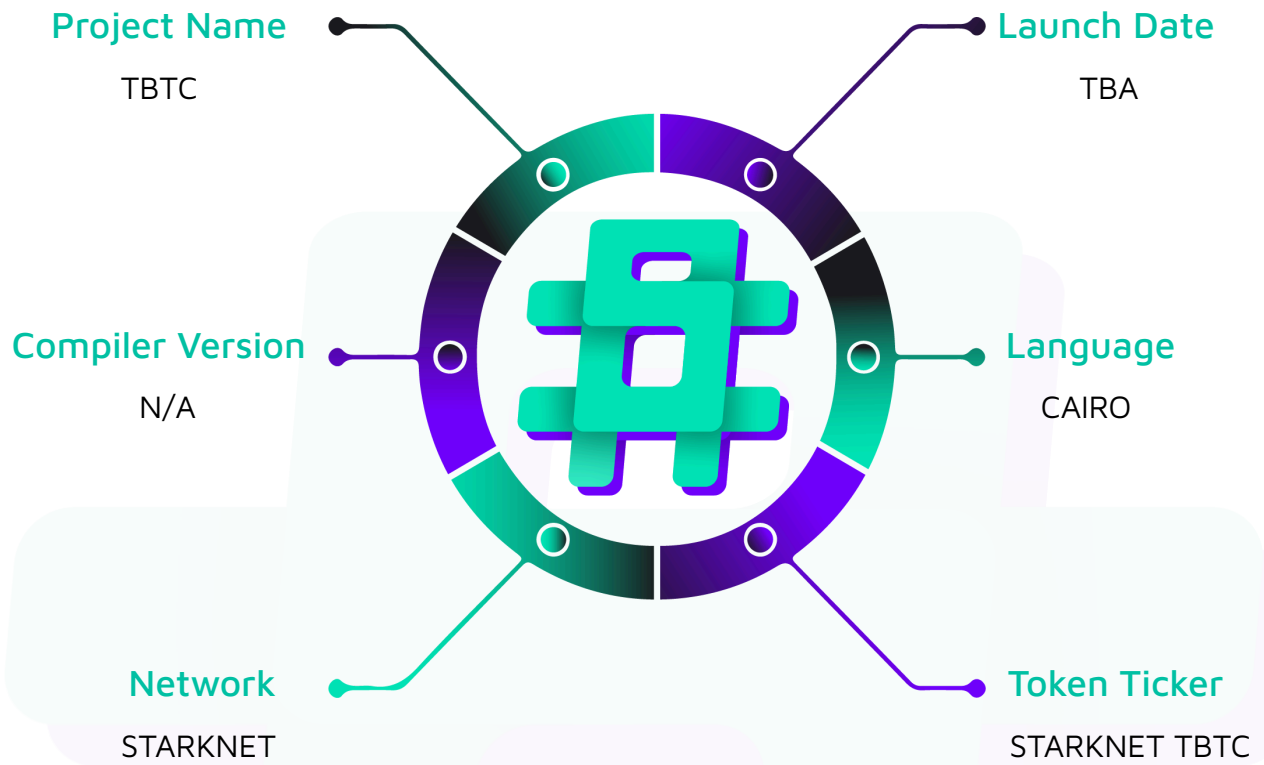
Project Type: Defi

Compiler Version: N/A

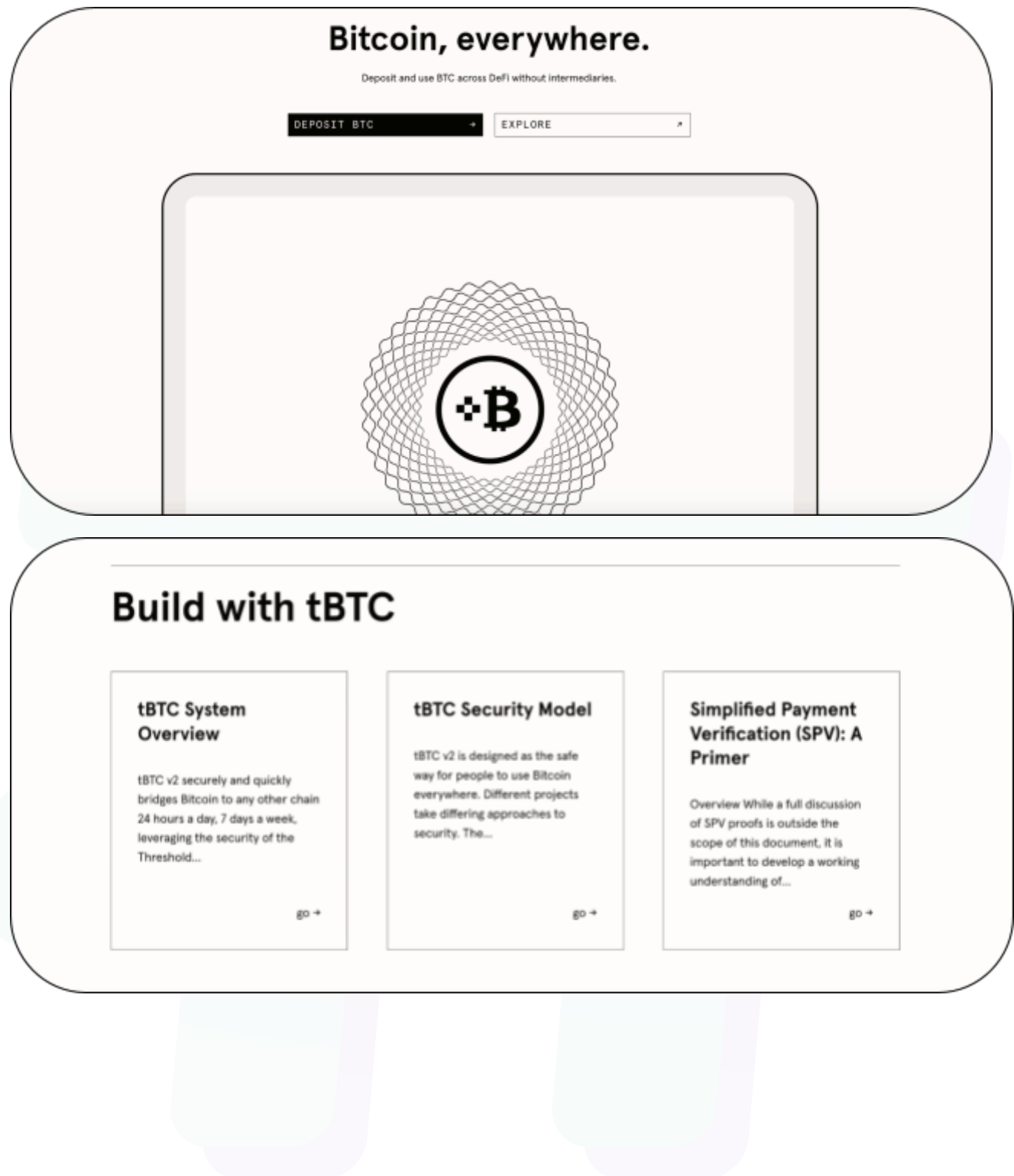
Website: <https://tbtc.network/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the cairo code within the tBTC project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	tBTC Smart Contracts
Platform	Starknet / Cairo
Audit Date	April, 2025
Contract 1	L2TBTC.cairo
Contract 1 MD5 Hash	98dc134d5ffb7289bbef472bc5aaf4bc
Audited GitHub Commit Hash	f5aa921b9cf10d6978a841713b7959c1c9431c87
Fix Review GitHub Commit Hash	93cb3ee292796b50214a7938e9ea2b330be4cc70

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

We initially identified some vulnerabilities that have since been addressed.

Hashlock found:

0 High severity vulnerabilities

0 Medium severity vulnerabilities

3 Low severity vulnerabilities

2 Gas Optimisations

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
L2BTC.cairo Allows users to: - Implements an ERC20-compatible token on Starknet - Utilizes role-based access control with owner, minters, and guardians - Restricts minting to authorized minters only - Allows token burning by any token holder - Includes a pause mechanism controlled by guardians - Supports contract upgradability via the owner - Provides token recovery functions for accidentally sent ERC20/ERC721 tokens - Implements ERC20Permit for gasless approval transactions - Uses OpenZeppelin components for security features - Emits events for role management and administrative actions - Stores lists of authorized minters and guardians	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the tBTC project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the tBTC project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] L2BTC#recover_ERC20 - Unrestricted self-token recovery

Description

The vulnerability exists in the `recover_ERC20` function, which allows the contract owner to transfer any ERC20 token from the contract to a specified recipient:

```
#[external(v0)]
fn recover_ERC20(
    ref self: ContractState,
    token: ContractAddress,
    recipient: ContractAddress,
    amount: u256
) {
    self.ownable.assert_only_owner();
    let erc20 = ERC20ABIDispatcher { contract_address: token };
    let success = erc20.transfer(recipient, amount);
    assert(success, 'ERC20 transfer failed');
}
```

This function lacks a critical check that would prevent the owner from extracting the contract's own tokens. Since the contract itself is an ERC20 token, passing the contract's address as the `token` parameter would allow the owner to transfer the tBTC to any address as well.

This is different from other well known protocols, for example Aave, Compound or Uniswap, where such a check is usually implemented.

Recommendation

Implement a specific check to prevent the contract's own tokens from being recovered.

Status

Acknowledged

[L-02] L2BTC#Ownable - Missing two-step ownership transfer pattern

Description

The contract uses OpenZeppelin's direct ownership transfer mechanism rather than a safer two-step ownership transfer pattern. This creates a security risk if the owner address is incorrectly specified during a transfer operation.

The contract inherits the Ownable component from OpenZeppelin which uses a direct transfer pattern where ownership can be reassigned in a single transaction. If the new owner address is incorrectly specified (e.g., a typo in the address or an address without corresponding private keys), the contract becomes permanently locked with no recovery mechanism.

The current pattern offers no safety mechanism to confirm that the new owner can actually access their account before finalizing the transfer. This design creates an unnecessary point of failure that could render the entire contract permanently inoperable due to a simple human error during administration.

Recommendation

Implement a two-step ownership transfer pattern that requires the new owner to accept ownership before the transfer is finalized. This pattern should include a pending owner storage variable to track the proposed new owner, a transfer initiation function that sets this pending value, and an acceptance function that can only be called by the pending owner to complete the transfer.

Status

Acknowledged

[L-03] L2BTC - Incomplete event emission for major functions

Description

The contract contains multiple state-changing functions that do not emit events, including `recover_ERC20`, `recover_ERC721`, and `upgrade`. While standard functions inherited from OpenZeppelin components likely emit appropriate events (e.g., Transfer events from burning/minting), these custom functions lack event emissions entirely.

For security-critical functions like token recovery and contract upgrades, event emission is essential for off-chain monitoring systems to detect potential exploitation or unexpected behavior. The lack of events for these operations creates a monitoring blind spot that could conceal malicious activities.

Recommendation

Implement appropriate events for all state-changing operations. New event structures should be defined for operations like token recovery and contract upgrades. For instance, an `ERC20Recovered` event should include the token address, recipient address, and amount, while an `ERC721Recovered` event should capture similar information for NFT recoveries.

Status

Resolved

Gas

[G-01] L2BTC - Interface-implementation discrepancy for `burn_from`

Description

The `burn_from` function is implemented in the contract but is not declared in the `IL2BTC` interface. This discrepancy might be problematic for developers attempting to integrate with the contract.

First, developers relying solely on the interface for understanding available functionality will be unaware of the `burn_from` capability, potentially missing an important contract feature.

Second, it violates interface-implementation consistency principles that are fundamental to good contract design.

This is primarily a code quality and developer experience issue rather than a direct security vulnerability.

Recommendation

Add the `burn_from` function to the `IL2BTC` interface to maintain consistency between the interface declaration and implementation. The function should be documented with appropriate natspec comments explaining its purpose, parameters, and behavior.

Status

Resolved

[G-02] L2BTC - Unused error constant

Description

The contract defines the constant `NOT_OWNER: felt252 = 'Not owner'` but never references it in the contract code. Instead, the contract uses the `assert_only_owner()` function from the `Ownable` component, which likely has its own error message.

While this is a minor issue, unused code can create confusion during maintenance and code reviews, potentially leading developers to mistakenly believe certain checks are implemented when they are not. Dead code also increases the cognitive load during code review and may complicate understanding of the error handling mechanisms in the contract.

Recommendation

Either remove the unused constant entirely from the codebase or modify relevant owner check functions to use this constant consistently.

Status

Resolved

QA

[Q-01] L2BTC - Inconsistent error message in pause

Description

The contract uses custom error messages across almost all functionalities. The only exception is the pause function, where in case of an error - text string is returned. This is problematic, as the specific event for this message is implemented, and it is called NOT_GUARDIAN.

For consistency, whole error messages should have similar format and structure.

Recommendation

We recommend changing the string used by the NOT_GUARDIAN error call.

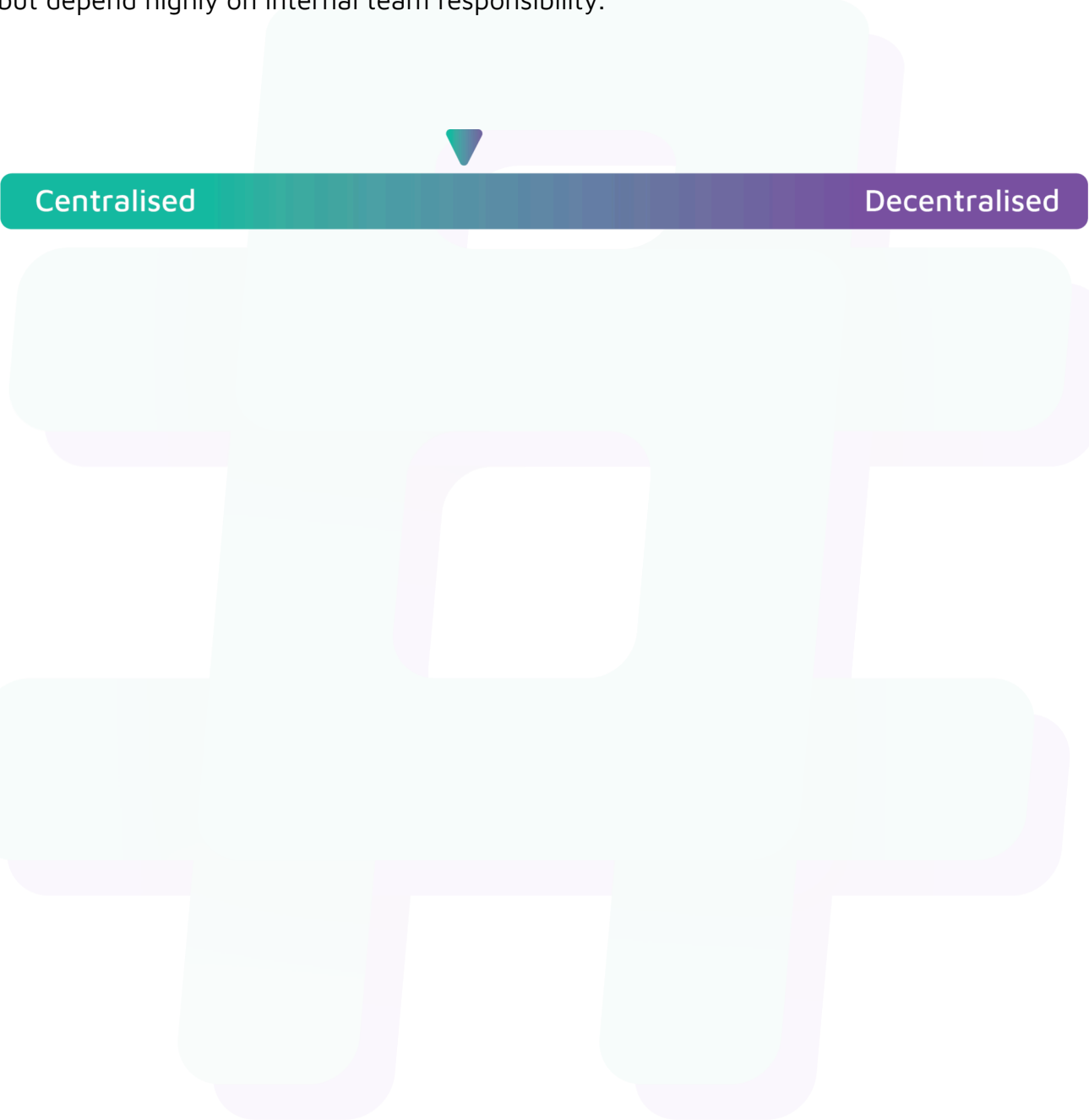
Status

Resolved

Centralisation

The tBTC project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the tBTC project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.